



ARCADE

Documentation

Sommaire

Introduction.....	2
Prérequis	2
Notre plan (architecture du projet)	3
Exécution du projet	4
Présentation des fonctions importantes du projet	5
Nous contacter	7

Introduction

Le projet Arcade vise à créer une plateforme de jeux vidéo multiplateforme en C++ permettant de développer, partager et jouer à des jeux rétro.

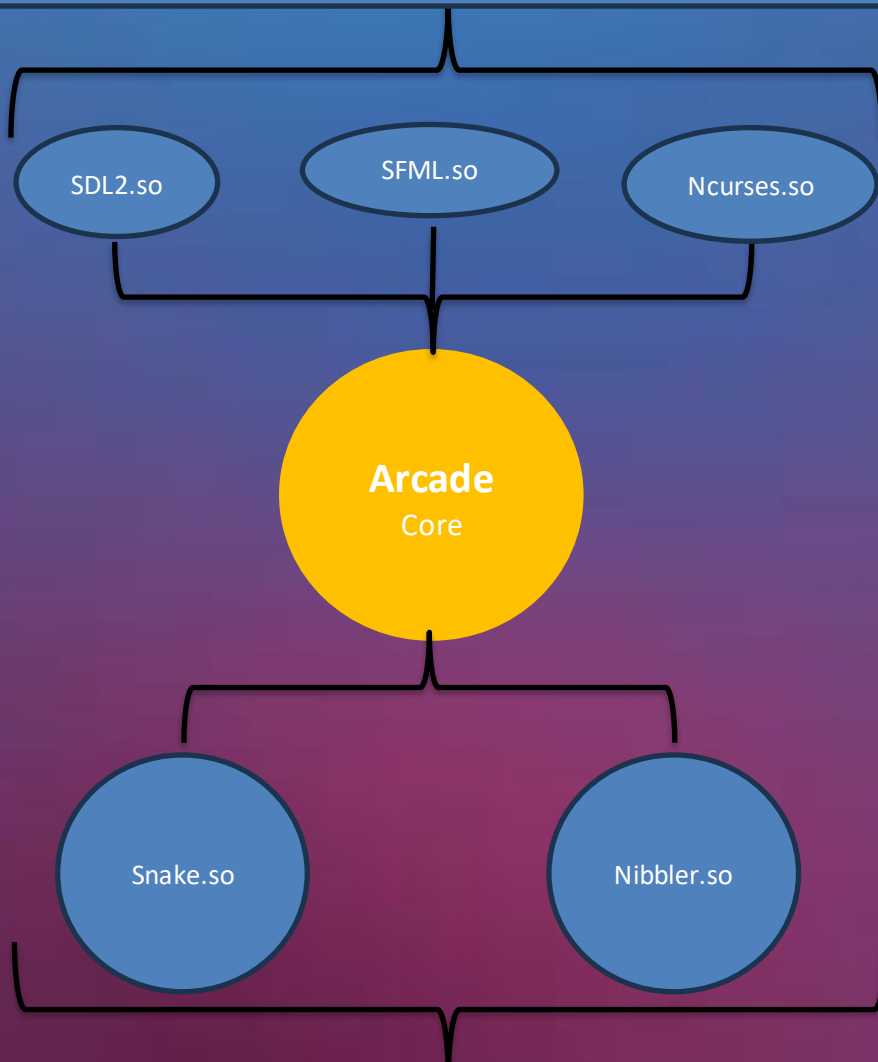
Prérequis

Pour avoir l'accès à notre projet Arcade

- Veuillez-vous assurer d'avoir les différentes libraires graphiques d'installer sur votre ordinateur(telles que la Ncurses, SDL2 et SFML).

Notre plan(architecture du projet)

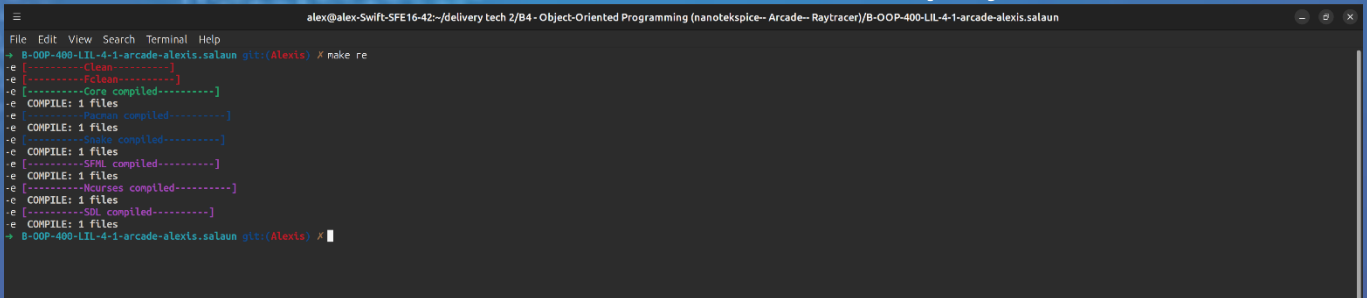
```
virtual int is_lib() = 0;    IDisplayModule: (interface des classes de librairies graphiques)
virtual void setWindow() = 0;    virtual void drawCircle(int x, int y, int radius, Color color) = 0;
virtual void drawWindow() = 0;    virtual void drawCircle(int x, int y, int radius, Color color) = 0;
virtual void clear() = 0;    virtual void drawText(int x, int y, std::string text, Color color) = 0;
```



```
virtual int is_lib() = 0;    IGame: (interface des classes de jeux)
virtual void setPlayer() = 0;    virtual int getPlace(int x, int y) = 0;
virtual void setPlace(int x, int y, int value) = 0;    virtual int getScore() = 0;
virtual void setScore(int score) = 0;    virtual bool checkCollision(int x, int y) = 0;
virtual int move() = 0;    virtual std::pair<int, int> getDir() = 0;
virtual void setDir(std::pair<int, int> dir) = 0;    virtual std::vector<std::vector<int>> getMap() = 0;
virtual bool isWon() = 0;
```

Exécution du programme

Au niveau de l'exécution, il vous suffit de faire :
Il vous suffit de faire `make re` dans votre terminal du projet.



```
alex@alex-Swift-SFE16-42:~/delivery tech 2/B4 - Object-Oriented Programming (nanotekspice- Arcade- Raytracer)/B-OOP-400-LIL-4-1-arcade-alexis.salaun
File Edit View Search Terminal Help
→ B-OOP-400-LIL-4-1-arcade-alexis.salaun git:(Alexis) X make re
-e [-----Clean-----]
-e [-----PClean-----]
-e [-----Core compiled-----]
-e COMPILER: 1 files
-e [-----Pacman compiled-----]
-e COMPILER: 1 files
-e [-----ncurses compiled-----]
-e COMPILER: 1 files
-e [-----SPML compiled-----]
-e COMPILER: 1 files
-e [-----Ncurses compiled-----]
-e COMPILER: 1 files
-e [-----SDL compiled-----]
-e COMPILER: 1 files
→ B-OOP-400-LIL-4-1-arcade-alexis.salaun git:(Alexis) X
```

Puis pour lancer les différents jeux dans la librairie souhaitée, il faut faire la commande suivante : `./arcade ./lib/arcade_[nom de la librairie].so`

→ B-OOP-400-LIL-4-1-arcade-alexis.salaun git:(Alexis) X `./arcade lib/arcade_ncurses.so`

→ B-OOP-400-LIL-4-1-arcade-alexis.salaun git:(Alexis) X `./arcade lib/arcade_sdl2.so`

→ B-OOP-400-LIL-4-1-arcade-alexis.salaun git:(Alexis) X `./arcade lib/arcade_sfml.so`

Touches importantes du projet :

Pour la sélection dans le menu :

- Veuillez entrer un nom d'utilisateur. Puis appuyer sur la touche Enter.
- Veuillez choisir une librairie et un jeu via la flèche du haut et du bas de votre clavier. Pressez la touche Enter pour sélectionner.
- Puis faire la même chose pour choisir le jeu que vous souhaitez entre le Snake ou le nibbler.

Pour le déplacement dans le jeu :

- Il faut appuyer sur les 4 flèches. (flèche du haut, du bas, droite, gauche)

Pour quitter le jeu et le menu :

- Il suffit d'appuyer sur le bouton Echap.

Présentation des fonctions importantes du projet

Dans le fichier IDisplayModule :

- is_lib : Retourne 1 si le fichier est une lib, une autre value sera comptait comme fichier non valide.
- setWindow : Pour initialiser la fenêtre avant de l'afficher
- drawWindow : Afficher la fenêtre
- clear : Effacer tout le ce qui est présent sur la fenêtre
- drawCircle : Dessiner un cercle, si mode = 1 alors le cercle sera centrer sur la fenêtre
- drawRect : Dessiner un rectangle, si mode = 1 alors le rectangle sera centrer sur la fenêtre
- drawTexte : Écrire du texte, si mode = 1 alors le texte sera centrer sur la fenêtre
- loadFont : Charger le fond pour la police d'écriture
- pollEvent : Gérer les événements qui peuvent se produire dans la fenêtre, selon les événements charge une certaine valeur dans une variable key (int key)
- closeWindow : Permet de fermer la fenêtre
- getKeys : Pour obtenir la valeur la variable key
- getWidth : Obtenir la largeur de la fenêtre
- getHeight : Obtenir la hauteur de la fenêtre

Dans le fichier IGame :

- **is lib** : Permet de différencier les librairies graphiques des jeux
- **setPlayer** : Initialise la position du joueur.
- **setPlace** : Initialise une position dans la map
- **setScore** : Initialise le score
- **setState** : Initialise l'état actuel du jeu
- **move** : Déplace le joueur
- **setDir** : Initialise la direction du joueur
- **getPlayer** : Retourne la position du joueur
- **getPlace** : Retourne la valeur d'une position de la map
- **getScore** : Retourne le score
- **getState** : Retourne l'état actuel du jeu
- **checkCollision** : Fonction générique de vérification des collisions
- **getDir** : Retourne la direction du joueur
- **getMap** : Retourne la map
- **isWon** : Vérifie si le joueur a gagné
- **isLost** : Vérifie si le joueur a perdu
- **loop** : Fonction de boucle principale du jeu appelé à chaque frame
- **getShapes** : Retourne les formes à afficher
- **setClock** : Initialise l'horloge à partir du temps réel
- **resetClock** : Réinitialise l'horloge
- **clearShapes** : Vide le cache des formes à afficher

Nous contacter

Si vous voulez plus d'informations, veuillez nous contacter par mail :

Alexis Salaun : alexis.salaun@epitech.eu

Julien Marcq : julien.marcq@epitech.eu

Luis Fernandes : luis.fernandes@epitech.eu